RouteRec: Behavior-Guided Sparse Routing for Sequential Recommendation

Anonymous Author(s)

Abstract

Sequential recommendation has advanced through stronger sequence models and richer use of behavioral context. These advances improve what the model encodes, but they still rely on the same shared transformation for every session prefix. Mixture of Experts (MoE) opens a different design choice: once conditional computation is available, what should define expert allocation in sessionized sequential recommendation?

We argue that raw interaction logs reveal recurring behavioral axes that create behavioral routing demand. Across datasets, sessionized inputs differ recurrently in tempo, transition ambiguity, memory regime, and exposure regime. **RouteRec** operationalizes these recurring behavioral axes as lightweight behavioral cues on a routing control path that stays separate from the sequential backbone. The router then performs hierarchical sparse expert allocation across macro, mid, and micro routing stages.

Under a shared sessionized temporal protocol across six public datasets, RouteRec achieves the best overall seen-target Avg. Rank (1.67 vs. 3.56 for the next-best baseline). The pattern also matches the proposed motivation: the gain is largest where local branching and memory variability are strong and where repeated-session context is sufficiently available, and it narrows where routing headroom is limited. Anonymous code: https://anonymous.4open.science/r/RouteRec/.

CCS Concepts

• **Information systems** → **Recommender systems**.

Keywords

sequential recommendation, sparse routing, conditional computation, behavioral cues, mixture of experts

ACM Reference Format:
Anonymous Author(s). 2026. RouteRec: Behavior-Guided Sparse Routing for Sequential Recommendation. In *Proceedings of the 20th ACM Conference on Recommender Systems (RecSys '26), September 28–October 2, 2026, Minneapolis, MN, USA*. ACM, New York, NY, USA, 18 pages.

1 Introduction

Sequential recommendation has advanced through stronger sequence encoders and richer use of behavioral context for next-item prediction [35]. These advances improve what the model encodes. Yet these models still apply the same shared transformation to every prefix, regardless of how differently sessions unfold. Once conditional computation enters the picture, this leaves an open question: what should define expert allocation in sessionized sequential recommendation?

RecSys '26, Minneapolis, MN, USA
2026.

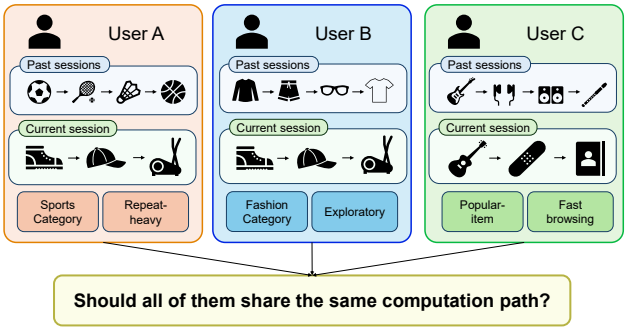


Figure 1: Two sessionized interaction sequences with different behavioral regimes still receive the same shared transformation in standard sequential recommendation.

Mixture of Experts (MoE) provides a direct answer in principle. It enables *conditional computation*, where only a data-dependent subset of experts is activated for each input instead of the full shared capacity [11, 14]. The router therefore determines the resulting *computation path* by selecting which experts are activated. MoE has already been applied in recommendation and sequential recommendation [21, 24, 33], but the core design question remains unresolved: in sessionized logs, which recurring behavioral variation should expert allocation follow?

Prior MoE work suggests how to answer it. One line of work improves routing mechanics, such as sparse activation, dispatch efficiency, and training stability [5, 30, 42]. Another line aligns routing with meaningful structure in the data: task relationships in multi-task learning [24, 33], domain and scenario identity in multi-domain recommendation [**?**], personalized priors in user-adaptive systems [**?**], and modality or temporal structure in multimodal sequential recommendation [1, 40]. The common thread is clear. Expert allocation works best when it is aligned with the source of variation that actually matters for specialization. This motivates the same question for sessionized sequential recommendation: along which *recurring behavioral axes* should expert allocation specialize?

Figure 1 illustrates the starting point. Sessionized interactions differ not only in which items appear, but also in how behavior unfolds. Some prefixes move at a steady pace, while others speed up or slow down sharply. Some branch into many plausible next steps, while others stay concentrated.

Some show strong repetition or carryover from earlier sessions, while others are shaped more by new exposure. These recurring patterns appear across datasets. They suggest that expert allocation should respond to behavioral regime, not only to item identity or hidden-state geometry.

We call this need *behavioral routing demand*: the extent to which recurring behavioral variation in sessionized logs creates a meaningful case for routed computation. To characterize that demand, we inspect raw interaction logs and identify four *recurring behavioral axes*: Tempo (pace and interval dynamics), Focus (concentration and switching), Memory (repetition and carryover), and Exposure (popularity level and drift). These axes are motivation-level descriptors. They describe where routing demand comes from, but they are not router inputs themselves. A related support condition is *context availability*, which asks whether enough cross-session evidence exists for macro-scope routing to act on.

RouteRec turns these recurring behavioral axes into lightweight *behavioral cues* computed directly from ordinary interaction logs, on a *routing control path* that is kept separate from the *sequential backbone*. This separation matters because hidden states are optimized for prediction and need not expose which behavioral property should define specialization. RouteRec therefore computes cue vectors at three temporal scopes: *macro* for cross-session history, *mid* for within-session evolution, and *micro* for short local transitions. These cue vectors drive a *hierarchical sparse router* that selects a small, behavior-dependent expert subset at each routing stage.

The experiments support this design. RouteRec achieves the best overall seen-target Avg. Rank across six datasets. The gain is largest on KuaiRec, Foursquare, and LastFM, where local branching and memory variability are strong and repeated-session context is sufficiently available. The margin narrows where those conditions are absent, which we interpret as limited *routing headroom*. We treat this empirical alignment as supporting evidence for the routing-design view, not as a post hoc argument.

The main contributions of this work are:

- We formulate MoE routing in sessionized sequential recommendation as a routing-design question: expert allocation should be aligned with recurring behavioral axes in interaction logs, not driven by hidden-state-driven routing alone.
- We propose **RouteRec**, which operationalizes this view through behavioral cue construction, coarse-to-fine multi-stage routing, and hierarchical sparse expert allocation on a routing control path separated from the sequential backbone.
- Under a shared sessionized temporal protocol across six public datasets, RouteRec achieves the best overall seen-target Avg. Rank, with the clearest gains on KuaiRec, Foursquare, and LastFM.

2 Related Work

2.1 Sequential Recommendation

Sequential recommendation predicts the next item from ordered user interactions [36]. Session-based recommendation is a closely related setting that groups interactions into short sessions and predicts from current-session context [12, 35]. Early models established core sequence backbones through factorized Markov transitions, convolutional architectures, and recurrent networks [10, 28, 34], and later models strengthened intent and transition modeling through attention and graph-based designs [19, 22, 37].

Transformer backbones such as SASRec and BERT4Rec became strong general references [15, 32]. More recent variants improve representation quality through contrastive objectives, frequency-aware bias, and state-space sequence modeling [4, 23, 26]. Side-information models such as FDSA and DIF-SR incorporate item attributes or timestamps to enrich the encoded representation [38, 41]. Across all these families, a shared transformation is applied to every prefix. This is the shared-path setup from which our routing question arises.

2.2 MoE in Recommendation

MoE introduces conditional computation by activating only a learned subset of experts for each input [11, 14]. A key insight from the multi-task learning literature is that expert assignment matters, not just the number of experts: MMoE and PLE show that aligning expert allocation with task relationships produces cleaner specialization than a fully shared pool [24, 33].

In sequential recommendation, expert-based designs have been applied to multimodal and temporal structure. M3SRec uses expert mixtures for multi-modal representation learning, and HM4SR applies hierarchical time-aware experts to short-video sequences [1, 40]. FAME uses expert-style factorization to model fine-grained preference facets [21]. These works establish that expert-based designs benefit sequential recommendation, but routing is still typically driven by hidden states or modality-specific encodings. The design question we focus on, which recurring behavioral axes in sessionized logs should define expert allocation, has not been directly addressed.

2.3 Sparse Routing Mechanics

A parallel line of MoE work focuses on routing mechanics: sparse activation for computational efficiency, load-balancing regularization, and training stability [5, 30, 42, 43]. RouteRec draws on this line for its sparse top-k selection and z-loss stabilization, but its primary contribution is on the routing-signal side: behavioral cues computed on a separate control path, rather than prediction-optimized hidden states, drive expert allocation.

3 Problem and Design Challenges

3.1 Problem Definition

For each user, let $\mathcal{S}_u = [s_1, \ldots, s_M]$ denote the chronologically ordered sequence of sessions, where each session $s_m = [x_{m,1}, \ldots, x_{m,L_m}]$ contains items interacted with during one continuous activity window. The task is *next-item prediction*: given the current-session prefix $x_{m,1:t}$ and the user's past sessions $[s_1, \ldots, s_{m-1}]$, predict the next item $x_{m,t+1}$.

Beyond item identities, interaction logs commonly record when each interaction occurred and what broad category the item belongs to, including timestamps and coarse grouping labels such as category, genre, or artist. Corpus-level popularity statistics can also be derived from the training data. We assume these fields are available in standard recommendation benchmarks; the method is designed to work best when they are present and to degrade gracefully when some are missing.

3.2 Design Challenges

RouteRec starts from a simple premise: recurring behavioral variation in sessionized logs can create *behavioral routing demand*, that is, a meaningful need for routed computation. Three design challenges follow from this premise. They determine whether behavioral routing demand can be translated into stable and behavior-dependent routes, or whether routing collapses back into a near-shared transformation.

C1. Cue portability versus informativeness. Behavioral routing demand must be read from signals that are widely available across datasets. In practice, this means item identities, timestamps, and sometimes a coarse grouping field such as category or genre. The difficulty is that such evidence is narrow. A cue built only from these fields may be too weak to differentiate routes, yet a cue that depends on richer metadata is unavailable in many benchmark settings. The first challenge is therefore to construct *behavioral cues* that remain portable while still carrying enough behavioral meaning to support different computation paths.

C2. Temporal scope of behavioral evidence. Behavioral routing demand is not temporally uniform. Cross-session repetition and carryover become visible only over multiple past sessions. Within-session concentration builds gradually over the current prefix. Local transition pressure appears only in the most recent interactions before the target. If all of this evidence is merged into a single routing signal, slow cross-session patterns and fast local dynamics mix together and lose their distinct roles. The second challenge is therefore to align routing decisions with the temporal scope at which each kind of behavioral evidence becomes reliable.

C3. Route commitment versus coverage. Even when routing reads meaningful cues at the right temporal scope, it can still fail to produce distinct routes. If weights spread broadly across experts, a repeat-heavy session and a fast exploratory session may activate almost the same mixture. The resulting model behaves little differently from a shared transformation. To avoid this collapse, routing must show *route commitment*: it must concentrate activation on a small, behavior-dependent expert subset. At the same time, it must preserve enough coverage to handle the full range of behavioral regimes seen in practice.

The next section addresses these challenges in the same order. It first identifies what observable evidence the router should read, then explains how that evidence should be separated by temporal scope, and finally shows how it is converted into committed sparse routes.

4 Proposed Method

RouteRec keeps two separate paths: a *sequential backbone* for next-item prediction, and a *routing control path* that reads behavioral cues from interaction sequences and selects experts. This separation keeps expert allocation aligned with behavioral routing demand rather than with what prediction-optimized hidden states happen to encode.

4.1 Overview

Figure 2 summarizes RouteRec at the framework level. The design follows the three challenges in Section 3.

C1. Behavioral Cue Construction. For C1, RouteRec fixes the routing evidence to simple metadata available before any model-specific transformation: item identities, timestamps, optional coarse grouping labels, and training-derived popularity statistics. These sources are portable across standard benchmarks and vary meaningfully from sample to sample. This makes them a natural basis for routing control. Section 4.2 organizes them into four *cue families*.

C2. Coarse-to-Fine Multi-Stage Routing. For C2, RouteRec separates routing by temporal scope. Cross-session repetition, within-session dynamics, and immediate local transitions each become reliable over different temporal ranges. A single routing view would blur these roles. Section 4.3 maps the three ranges to macro, mid, and micro routing stages.

C3. Hierarchical Sparse Expert Allocation. For C3, the cue families have different behavioral meanings, and the family that matters most varies across inputs. A flat expert pool makes these differences hard to use. All cues compete in one undifferentiated selection step, and behaviorally different sessions can still end up on the same route. RouteRec instead groups experts by cue family and applies sparse top-k selection at both levels, so cue differences produce distinct computation paths. Section 4.4 details this design.

4.2 Behavioral Cue Construction

The routing control path computes behavioral cues directly from interaction sequences, without passing through the sequential backbone. Given a window of interactions $\mathcal{R}$, a shared summary function $\mathcal{A}(\cdot)$ produces a fixed-size *cue vector* $\mathbf{f} = \mathcal{A}(\mathcal{R})$ consisting of scalar statistics derived from raw sequence fields.

The four recurring behavioral axes from Section 1, Tempo, Focus, Memory, and Exposure, define the structure of this cue

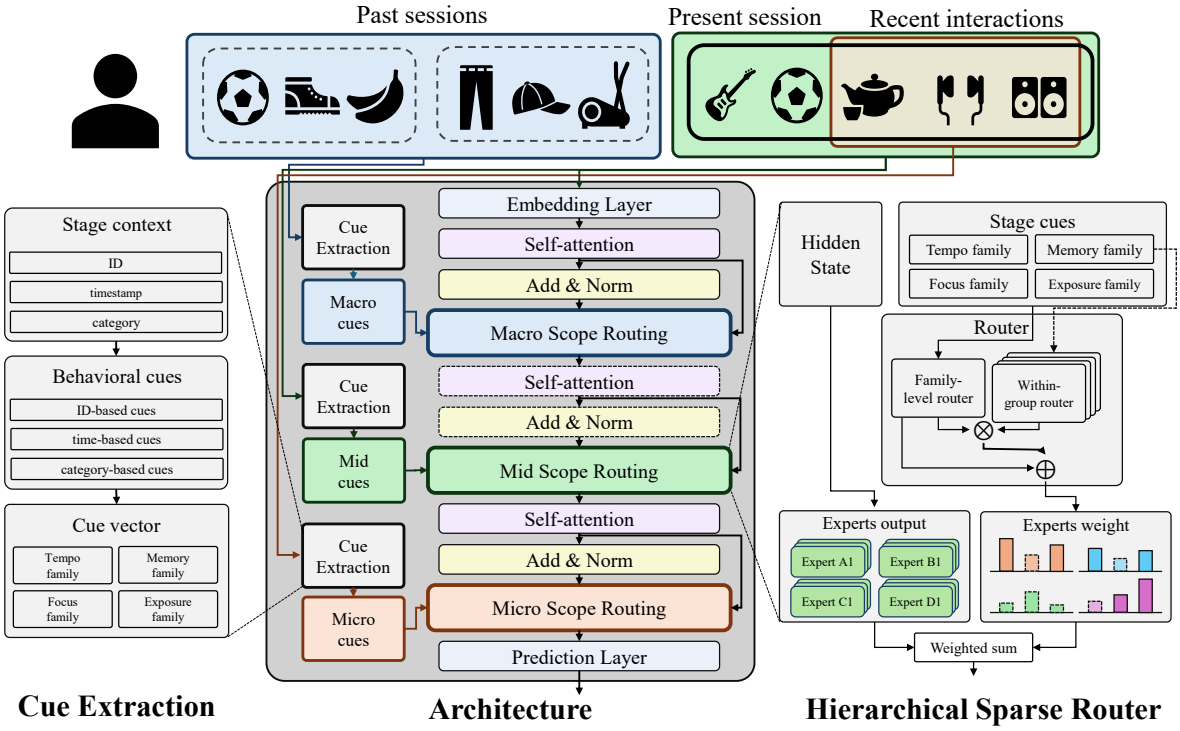


Figure 2: RouteRec overview. *Left*: raw interaction fields are summarized into four cue families at each temporal scope. *Center*: the sequential backbone inserts macro, mid, and micro routing stages between self-attention blocks so that slower cross-session evidence and faster local evidence act at different points. *Right*: hierarchical sparse expert allocation first selects the most relevant cue families and then selects experts within those families to form the active computation path.

vector. Rather than starting from an arbitrary signal bank, we identify metadata that directly exposes each axis and varies meaningfully from sample to sample: item identities reveal repetition and carryover (Memory), timestamps reveal pace and interval shifts (Tempo), coarse grouping labels reveal concentration and switching (Focus), and training-derived popularity statistics reveal exposure level and drift (Exposure). These sources are available in nearly all standard sessionized benchmarks and can be computed before any model transformation.

Each axis is operationalized as a compact family of scalar summaries. The *Tempo* family captures interaction pace and interval dynamics. The *Focus* family captures concentration and switching patterns over the coarsest grouping structure available, falling back to a fixed missing-signal rule when no grouping field is present (see Appendix A). The *Memory* family captures repetition, recurrence, and carryover across items. The *Exposure* family captures popularity level and drift within the window. Together, the four families cover the main behavioral distinctions visible in sessionized logs without expanding the router into a full feature-engineering module.

The same four-family template is applied at all three temporal windows (macro, mid, micro), described next. Representative cue definitions are in Table 5; full specifications are in Appendix A. The robustness of behavior-guided routing when individual cue sources are missing is evaluated separately in Appendix H.

4.3 Coarse-to-Fine Multi-Stage Routing

The next question is *when* behavioral evidence should affect routing. Different cues become reliable at different temporal scales. Repetition and carryover need multiple past sessions. Session concentration and pace need a growing within-session prefix. Immediate transition pressure is visible only in the most recent few interactions. A single routing view would mix these signals and blur their roles. RouteRec therefore places a dedicated routing stage at each temporal scope, each reading behavioral cues computed from its own window of history. For session s_m at prediction target t, the three windows are:

$$\mathcal{R}_t^{\text{macro}} = [s_{\max(1,m-H)}, \ldots, s_{m-1}], \tag{1}$$

$$\mathcal{R}_t^{\text{mid}} = [x_{m,1}, \ldots, x_{m,t-1}], \tag{2}$$

$$\mathcal{R}_t^{\text{micro}} = [x_{m,\max(1,t-w)}, \ldots, x_{m,t-1}], \tag{3}$$

where H is the number of past sessions and w the local window size. Applying $\mathcal{A}(\cdot)$ to each window gives stage-specific cue vectors $\mathbf{f}_t^{(\text{macro})}$, $\mathbf{f}_t^{(\text{mid})}$, $\mathbf{f}_t^{(\text{micro})}$.

Each stage is tied to the temporal role of its window. Macro reads slower cross-session tendency. Mid tracks how the current session develops. Micro reads the immediate transition context near the target. This keeps each routing stage tied to evidence that is actually reliable at its own scale.

The three stages are interleaved with self-attention blocks in the backbone. This keeps routing tied to the temporal role of each stage without expanding the architecture into many competing pathways. In the main configuration:

$$\underbrace{\text{self-attn}}_{\text{contextualize}} \rightarrow \text{macro} \rightarrow \text{mid} \rightarrow \underbrace{\text{self-attn}}_{\text{refresh}} \rightarrow \text{micro}.$$

Macro and mid are both *sequence-level* decisions. Cross-session tendency and within-session dynamics are properties of the whole session, not of individual positions. A repeat-heavy session stays repeat-heavy throughout, not only at particular steps. It therefore makes sense to compute one routing decision per sequence (from averaged cue vectors) and broadcast it uniformly. Micro, by contrast, is *position-level*: local transition patterns differ at each step, so micro computes a separate routing decision per position. A second self-attention block is placed before micro so that the local router sees a refreshed token state immediately before prediction. Full formulations are in Appendix A.

Section 4.4 describes how expert selection within each stage enforces route commitment.

4.4 Hierarchical Sparse Expert Allocation

The final question is whether cue differences at the router input actually produce different computation paths at the output. Even after meaningful cues are computed at the right temporal scope, routing can still fail. If weights spread across all experts, a repeat-heavy session and a fast-exploratory session may activate nearly the same mixture. The result behaves little differently from a shared transformation. Two problems underlie this risk. First, the cue families carry different behavioral meanings, but a flat expert pool treats all cues identically in one undifferentiated selection step. Second, soft mixing without sparsity lets cue variation average out rather than drive distinct routes.

RouteRec addresses both problems through *route commitment*. It concentrates activation on a small, behavior-dependent expert subset by organizing the E experts into G *expert groups* of C experts each ($E = GC$), where each group is aligned with one cue family (Tempo, Focus, Memory, or Exposure). The router first selects which cue family drives computation, then selects specific experts within that family. Sparse top-k selection at both levels ensures that two inputs with different cues cannot converge to the same route through soft averaging.

Router formulation. The router takes the cue vector $\mathbf{f}_t^{(s)}$ as input and produces two distributions via learned linear projections: a family-level distribution $p_{t,g}^{(s)}$ over expert groups, and a within-group distribution $r_{t,g,c}^{(s)}$ over experts inside each group. Their product $\pi_{t,g,c}^{(s)} = p_{t,g}^{(s)} \cdot r_{t,g,c}^{(s)}$ gives the joint weight for expert (g, c). The factored form matters because it lets the model use cue semantics explicitly: first decide which cue family is most relevant, then select the expert that should respond within that family. The same router structure is shared across all three stages; stages differ only in which cue vector they receive.

Sparse selection. Route commitment is enforced by hard top-k selection at both levels: only the top-k_g expert groups and the top-k_e experts within each are active, with their weights renormalized. All other experts contribute nothing to the output. Let $\text{TopK}(\mathbf{v}, k)$ be the index set of the k largest entries of $\mathbf{v}$; the sparse group weights $\bar{p}_{t,g}^{(s)}$ and within-group weights $\bar{r}_{t,g,c}^{(s)}$ are:

$$\bar{p}_{t,g}^{(s)} = \frac{p_{t,g}^{(s)}\, \mathbf{1}[g \in \text{TopK}(\mathbf{p}_t^{(s)}, k_g)]}{\sum_{g'} p_{t,g'}^{(s)}\, \mathbf{1}[g' \in \text{TopK}(\mathbf{p}_t^{(s)}, k_g)]}, \tag{4}$$

$$\bar{r}_{t,g,c}^{(s)} = \frac{r_{t,g,c}^{(s)}\, \mathbf{1}[c \in \text{TopK}(\mathbf{r}_{t,g,:}^{(s)}, k_e)]}{\sum_{c'} r_{t,g,c'}^{(s)}\, \mathbf{1}[c' \in \text{TopK}(\mathbf{r}_{t,g,:}^{(s)}, k_e)]}. \tag{5}$$

The stage output sums only over active experts ($\hat{\pi}_{t,g,c}^{(s)} = \bar{p}_{t,g}^{(s)}\, \bar{r}_{t,g,c}^{(s)}$):

$$\tilde{\mathbf{h}}_t^{(s)} = \sum_{g,c} \hat{\pi}_{t,g,c}^{(s)}\, \text{Expert}_{g,c}^{(s)}(\mathbf{h}_t^{\text{in},(s)}). \tag{6}$$

Sparsity here is about commitment, not computational efficiency: it ensures that two sessions with different cues cannot end up on the same route just because the soft weights happen to average out the same way. Dense soft mixing, where every expert contributes a small nonzero weight, is evaluated as an ablation in Section 5.4. Configuration values (G, C, k_g, k_e) are in Appendix A.

4.5 Training Objective

RouteRec is trained end-to-end with one prediction loss and two routing regularizers. Maintaining route commitment requires the router to stay cue-sensitive throughout training. The prediction loss alone cannot guarantee this: cue-driven routing can degenerate in two distinct ways that the prediction loss does not penalize.

Prediction loss. The main objective is standard next-item cross-entropy. Let $\mathbf{z}$ be the final logit vector over all items and y the target item:

$$\mathcal{L}_{\text{CE}} = -\log \frac{\exp(z_y)}{\sum_j \exp(z_j)}.$$

This loss drives the model to rank the correct next item highly, but it provides no direct signal about whether the routing is cue-sensitive or not.

Table 1: Seen-target test results under the shared sessionized temporal protocol. Datasets marked with † are sampled processed subsets. Each method reports the run with the best validation performance under the shared selection rule. Lower Avg. Rank is better.

Dataset	Metric	SASRec	GRU4Rec	TiSASRec	FEARec	DuoRec	BSARec	FAME	DIF-SR	FDSA	RouteRec
KuaiRec†	HR@10	0.3284	0.3061	0.3216	0.3513	0.3468	0.3359	0.3307	0.2333	0.3453	**0.3669**
	NDCG@10	0.3177	0.2589	0.2889	0.3286	0.3215	0.3209	0.3188	0.2229	0.3197	**0.3467**
	MRR@20	0.3150	0.2462	0.2802	0.3227	0.3153	0.3170	0.3160	0.2201	0.3133	**0.3425**
Foursquare	HR@10	0.3049	0.2180	0.2976	0.3054	0.3195	0.2383	0.2417	0.3032	0.3042	**0.3215**
	NDCG@10	0.1929	0.1476	0.1926	0.1925	0.1970	0.1609	0.1604	0.2008	0.2014	**0.2042**
	MRR@20	0.1624	0.1286	0.1635	0.1613	0.1631	0.1393	0.1378	0.1723	**0.1726**	0.1715
LastFM†	HR@10	0.3775	0.3109	0.3715	0.3876	0.3850	0.3475	0.3359	0.3593	0.3762	**0.3965**
	NDCG@10	0.3097	0.2676	0.3029	0.3187	0.3066	0.3044	0.2972	0.3169	0.3247	**0.3307**
	MRR@20	0.2897	0.2558	0.2831	0.2981	0.2833	0.2915	0.2861	0.3051	0.3103	**0.3113**
Retail Rocket	HR@10	0.5321	0.5048	0.5131	0.5487	0.5478	0.4655	0.4612	0.4322	0.5343	**0.5608**
	NDCG@10	0.3907	0.3674	0.3755	0.3998	0.3994	0.3957	0.3943	0.3863	0.3848	**0.4148**
	MRR@20	0.3507	0.3290	0.3368	0.3581	0.3576	**0.3745**	0.3738	0.3709	0.3427	0.3737
Beauty	HR@10	0.1324	0.0840	0.1235	0.1404	0.1347	0.0726	0.0497	0.1165	0.1318	**0.1561**
	NDCG@10	0.0800	0.0457	0.0831	0.0797	0.0793	0.0504	0.0303	0.0811	0.0918	**0.0948**
	MRR@20	0.0659	0.0361	0.0735	0.0634	0.0662	0.0450	0.0250	0.0716	**0.0822**	0.0794
ML-1M	HR@10	0.1423	0.1456	0.1448	0.1490	0.1555	0.1374	0.1533	0.0669	**0.1606**	0.1568
	NDCG@10	0.0717	0.0778	0.0756	0.0736	0.0810	0.0723	0.0818	0.0331	**0.0833**	0.0807
	MRR@20	0.0555	0.0621	0.0600	0.0559	0.0641	0.0570	0.0651	0.0256	**0.0655**	0.0631
Avg. Rank↓		**6.06**	**8.67**	**6.61**	**4.22**	**4.17**	**6.61**	**6.72**	**6.72**	**3.56**	**1.67**

Route consistency. The first failure mode is routing collapse: the router ignores behavioral cues and assigns all sessions to the same route. The model then behaves like a shared feed-forward block. Route consistency pushes against this by requiring sessions with similar behavioral cues to produce similar routing distributions. For each stage s and sample i, let $\bar{\mathbf{f}}_i^{(s)} = \frac{1}{L_i}\sum_t \mathbf{f}_{i,t}^{(s)}$ be the session-level cue average and $\bar{\boldsymbol{\pi}}_i^{(s)} = \frac{1}{L_i}\sum_t \boldsymbol{\pi}_{i,t}^{(s)}$ the corresponding average routing distribution. We find each sample's nearest cue neighbor in the mini-batch (pair set $\mathcal{P}_s$) and penalize the divergence between their routing distributions:

$$\mathcal{L}_{\text{cons}} = \frac{1}{|\mathcal{T}|}\sum_{s\in\mathcal{T}} \frac{1}{|\mathcal{P}_s|} \sum_{(i,j)\in\mathcal{P}_s} \text{JSD}\left(\bar{\boldsymbol{\pi}}_i^{(s)}, \bar{\boldsymbol{\pi}}_j^{(s)}\right).$$

This encourages the router to produce routing distributions that reflect the structure of the cue space, not just whatever pattern minimizes the prediction loss.

Routing logit stabilization. The second failure mode is subtler: even when the router tries to be cue-sensitive, the pre-softmax logits can grow very large during training. When this happens, the softmax saturates and routing collapses to a near-hard assignment. Small cue differences then become invisible to the router, so the cue vector stops mattering in practice even though the router nominally reads it. We address this with z-loss [43], which penalizes the scale of the pre-softmax expert logits $a_{i,t,g,c}^{s}$ to keep the routing distribution in a well-behaved range where cue differences can produce meaningfully different outputs:

$$\mathcal{L}_z = \frac{1}{|\mathcal{T}|}\sum_{s\in\mathcal{T}} \frac{1}{N_s} \sum_{(i,t)\in\mathcal{V}_s} \left(\log \sum_{g,c} \exp a_{i,t,g,c}^{s}\right)^2,$$

where $\mathcal{V}_s$ is the set of valid positions and $N_s = |\mathcal{V}_s|$. The final objective is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{cons}}\,\mathcal{L}_{\text{cons}} + \lambda_z\,\mathcal{L}_z.$$

We do not include a load-balancing term. Classical MoE training often penalizes unequal expert usage to prevent collapse onto a small subset. Here the goal is the opposite: behaviorally distinct sessions should activate different, committed routes rather than distributing traffic evenly. Route consistency and logit stabilization together are sufficient for stable, cue-aligned specialization.

5 Experiments

5.1 Experimental Setup

Datasets and protocol. We evaluate on six public datasets: KuaiRec [7], Foursquare [39], LastFM [3], Retail Rocket [29], Beauty [25], and ML-1M [9]. All datasets are preprocessed under a shared sessionized temporal protocol: interactions are sorted chronologically, grouped into sessions by inactivity threshold, and filtered to remove short sessions and low-support items. The resulting sessions are split chronologically into 70%/15%/15% train/validation/test partitions, and the prediction task is next-item prediction for the last item of

each held-out session. KuaiRec and LastFM are large; we use 20% and 3% random subsets respectively, and all baselines are evaluated on the same data. Full preprocessing details and dataset statistics are in Appendix C.

Metrics. We report Hit Rate (HR@k), Normalized Discounted Cumulative Gain (NDCG@k), and Mean Reciprocal Rank (MRR@k) at $k \in \{5, 10, 20\}$. All evaluations use the seen-target setting, where the candidate set consists of items observed during training. This is standard practice for ID-based sequential recommendation, which cannot score items outside the training vocabulary [18]. The primary reported metrics are HR@10, NDCG@10, and MRR@20.

Baselines. We compare against nine baselines in three groups. *Conventional sequential models*, namely GRU4Rec [10], SASRec [15], and TiSASRec [20], cover recurrent and self-attention backbones and represent the standard shared-computation baseline. *Representation-enhanced models*, namely DuoRec [26], FEARec [4], and BSARec [31], improve over the standard backbone through contrastive regularization or frequency-aware encoding, but still apply the same transformation to every session. *Side-information and MoE models*, namely FDSA [41], DIF-SR [38], and FAME [21], incorporate external item features or expert decomposition; they are the closest comparison to RouteRec in that they go beyond a purely shared backbone. All methods use the same processed data, the same optimization family, matched early stopping, and the same model-selection rule. Tuning details are in Appendix C.

5.2 Main Comparison

Table 1 reports the main seen-target comparison under the shared protocol. We use Avg. Rank as the primary summary metric because absolute score scales differ substantially across datasets. RouteRec achieves the best overall Avg. Rank (1.67 vs. 3.56 for FDSA, the next-best baseline).

The routing-design motivation predicts that gains should be larger where the recurring behavioral axes are most active and repeated-session context is available. Table 2 tests this reading. It pairs each dataset's HR@10 gain with four indicators computed from raw training logs before any model training. Because the indicators are derived from logs independently of the model, a correlation between indicator levels and gain provides evidence that the recurring behavioral axes carry useful routing signal in the raw data itself, rather than simply reflecting patterns the model has already internalized.

Each indicator captures one recurring behavioral axis. Temporal volatility is the variability of session length across users. Transition branching measures how widely the next action branches from a similar local state. Memory variability measures how uneven repetition behavior is across sessions. Context availability summarizes how much repeated-session evidence is present for macro-scope routing. All four are scale-free dataset-level summaries derived from raw logs. The Exposure axis (popularity concentration) is included in the full axis table in Appendix B; its correlation with gain is weaker across these six datasets, so it is not the primary driver of the pattern here.

Table 2: Raw-log behavioral indicators and RouteRec HR@10 gain over the best baseline. All indicator values are computed from training logs before model training. Full definitions are in Appendix B.

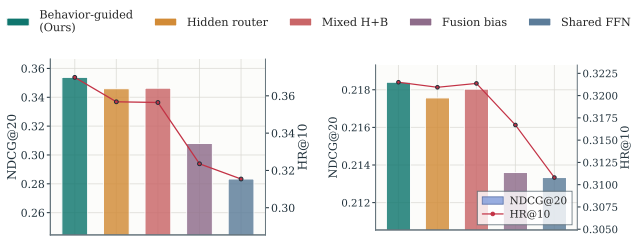
Dataset	Temporal volatility	Transition branching	Memory variability	Context availability	HR@10 gain
KuaiRec	0.887	0.903	2.214	0.933	+4.44%
Foursquare	0.944	0.688	1.961	0.988	+0.63%
LastFM	0.739	0.630	2.036	0.990	+2.30%
Retail Rocket	0.945	0.744	0.858	0.056	+2.21%
Beauty	0.694	0.843	0.000	0.059	+11.18%
ML-1M	0.381	0.839	0.000	0.264	-2.37%

The gain pattern is broadly directional with these indicators. KuaiRec combines the strongest transition branching, the highest memory variability, and high context availability; it achieves the strongest and most consistent overall gains across all reported metrics. Foursquare has strong temporal volatility and high context availability, and also benefits, though more modestly. LastFM has weaker local branching but similarly high memory variability and near-maximal context availability, yielding a clear positive gain.

The remaining three datasets are instructive boundary cases. Retail Rocket has strong temporal volatility but near-zero context availability, which limits broader-scope routing and keeps the HR@10 margin positive but contained. Beauty has the highest HR@10 gain in absolute terms despite near-zero context availability; its gain is concentrated in HR@10 and the other two metrics are more mixed (see Table 1), suggesting that strong local transition branching supports short-horizon routing even when macro-routing headroom is low. ML-1M is the one dataset where RouteRec trails: temporal volatility is the lowest among the six, memory variability is zero, and context availability is limited. Together, these conditions point to low routing headroom overall.

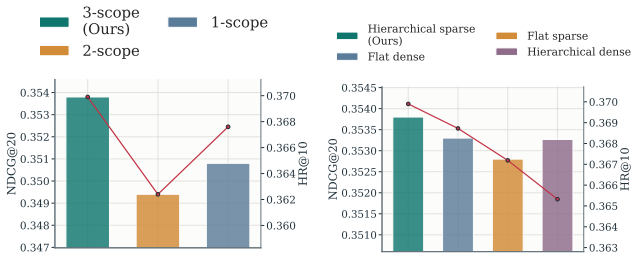
These comparisons are not causal claims. The six-dataset comparison is too small for strong statistical inference, and the indicators are directional descriptors rather than a predictive score. What they show is that the relative gain pattern is broadly coherent with the behavioral routing conditions that motivated RouteRec. A within-dataset breakdown by session length and target frequency on KuaiRec is in Appendix D.

The closest baselines are FDSA, DIF-SR, and FAME, each of which goes beyond a shared backbone through side information or expert decomposition. FDSA and DIF-SR incorporate external item features but apply a shared transformation to all sessions. FAME introduces expert decomposition but routes from hidden states, tying routing decisions to the prediction objective. RouteRec's advantage over this group is largest on the datasets where recurring behavioral axes are most active, which is consistent with behavioral cues providing a cleaner routing signal than hidden states alone.



(a) KuaiRec (b) Foursquare

Figure 3: Routing-input ablations on KuaiRec and Foursquare.



(a) Temporal scopes (b) Routing organization

Figure 4: Routing-design ablations on KuaiRec: (a) temporal scopes and (b) routing organization.

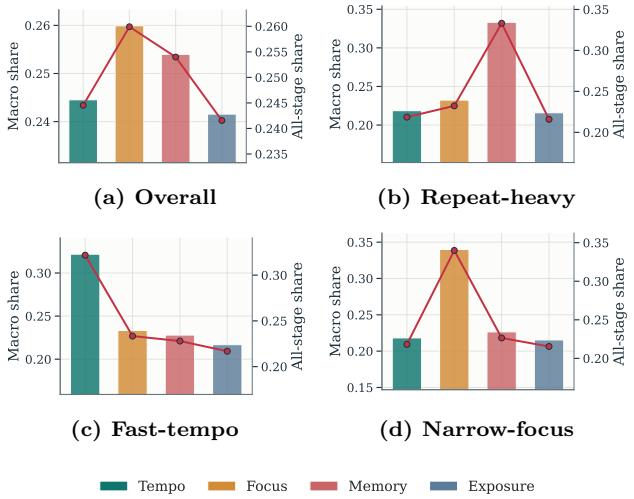
5.3 Routing Control

The overall comparison shows that RouteRec performs best on average, but does not directly isolate what makes the routing signal effective. The next question is whether behavioral cues are actually a better routing signal than hidden states. Hidden states are optimized for next-item prediction, not for reflecting behavioral character, and routing from them ties the control path to the very representation it is meant to influence.

Figure 3 compares five routing-input variants on KuaiRec and Foursquare. Hidden-only routing already improves over the shared FFN baseline, and mixing hidden states with behavioral cues adds further gain. The behavior-guided router, which keeps the routing control path fully separate from the backbone, yields the most consistent results across both datasets. The main advantage therefore comes from separating the two paths, not merely from enriching the router input.

5.4 Temporal Scope and Routing Design

Given that behavioral cues outperform hidden states as the routing signal, the next question is whether the three-scope layout and hierarchical sparse expert allocation are both necessary. RouteRec addresses C2 (Section 3) by distributing routing across macro, mid, and micro temporal scopes, and C3 by enforcing hierarchical sparse expert selection within each stage. We test both design choices on KuaiRec.



(a) Overall (b) Repeat-heavy

(c) Fast-tempo (d) Narrow-focus

Figure 5: Family-weight routing profiles for four user subsets on KuaiRec: (a) the full test population as an anchor and (b–d) subsets selected for elevated repeat intensity, fast interaction pace, and narrow topical focus.

The result is clear in both panels. In panel (a), the full three-scope design performs best. Removing any single scope degrades performance, with the largest drop when the macro scope is absent: cross-session behavioral tendency cannot be recovered from within-session evidence alone. In panel (b), both flat routing and dense activation underperform the hierarchical sparse design under a comparable expert budget, showing that family-level structure and sparse commitment each contribute independently to route quality.

5.5 Routing Profiles

The preceding ablations show that the design components are necessary. A complementary question is whether the router actually responds to behavioral differences between sessions. If expert allocation is behavior-guided, sessions with different behavioral character should produce different family-weight profiles at each routing stage.

Figure 5 shows family-weight profiles at each routing stage for four user subsets on KuaiRec. Panel (a) is the full test population and serves as an anchor. Panels (b)–(d) each isolate users with one elevated behavioral property: repeat-heavy users concentrate along the Memory axis, fast-tempo users along the Tempo axis, and narrow-focus users along the Focus axis.

The profiles shift in ways that are consistent with the recurring behavioral axes motivating RouteRec. Repeat-heavy users produce higher Memory family weight at the macro scope, where cross-session repetition evidence accumulates. Fast-tempo users show elevated Tempo family weight, most visible at mid and micro scope where pace and interval variation are freshest. Narrow-focus users produce a stronger Focus family weight, concentrated in the within-session scope

where topical consistency is most apparent. These shifts suggest that the router has learned to respond to behavioral cue content rather than treating all sessions uniformly.

6 Conclusion

Sessionized interaction logs exhibit recurring behavioral variation along axes such as session pace, local transition ambiguity, repetition structure, and popularity exposure. This variation creates *behavioral routing demand*: a meaningful case for routing inputs through different computation paths rather than a single shared transformation. RouteRec addresses this by operationalizing four recurring behavioral axes as lightweight cues on a routing control path kept separate from the sequential backbone.

Across six datasets, RouteRec achieves the best overall seen-target Avg. Rank. The gain pattern is directionally consistent with the routing motivation: the largest improvements appear on KuaiRec, Foursquare, and LastFM, where local branching, memory variability, and repeated-session context are strongest. The ablations confirm that all three temporal scopes contribute, hierarchical sparse selection outperforms flat and dense alternatives, and behavior-guided routing outperforms hidden-state-driven routing. The routing profile analysis further shows that family-weight allocations shift in directions aligned with behavioral differences across user subsets.

The broader takeaway is that once conditional computation is available, what should drive expert allocation becomes a genuine modeling choice. RouteRec makes that choice explicit by grounding routing in recurring behavioral structure visible in raw interaction logs. The results show that this alignment produces consistent gains over strong shared-computation baselines across diverse domains and session regimes.

References

[1] Shuqing Bian, Xingyu Pan, Wayne Xin Zhao, Jinpeng Wang, Chao Wang, and Ji-Rong Wen. 2023. Multi-modal Mixture of Experts Representation Learning for Sequential Recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*. 110–119. doi:10.1145/3583780.3614978

[2] Simone Borg Bruun, Maria Maistro, and Christina Lioma. 2022. Learning Recommendations from User Actions in the Item-Poor Insurance Domain. In *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys)*. Association for Computing Machinery, New York, NY, USA, 113–123. doi:10.1145/3523227.3546775

[3] Oscar Celma. 2010. *Music Recommendation and Discovery in the Long Tail*. Springer. doi:10.1007/978-3-642-13287-2

[4] Xinyu Du, Huanhuan Yuan, Pengpeng Zhao, Jianfeng Qu, Fuzhen Zhuang, Guanfeng Liu, Yanchi Liu, and Victor S. Sheng. 2023. Frequency Enhanced Hybrid Attention Network for Sequential Recommendation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. 78–88. doi:10.1145/3539618.3591689

[5] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39. https://jmlr.org/papers/v23/21-0998.html

[6] B. G. Galuzzi, I. Giordani, A. Candelieri, R. Perego, and F. Archetti. 2020. Hyperparameter Optimization for Recommender Systems Through Bayesian Optimization. *Computational Management Science* 17, 4 (2020), 495–515. doi:10.1007/s10287-020-00376-3

[7] Chongming Gao, Shijun Li, Wenqiang Lei, Jiawei Chen, Biao Li, Peng Jiang, Xiangnan He, Jiaxin Mao, and Tat-Seng Chua. 2022. KuaiRec: A Fully-Observed Dataset and Insights for Evaluating Recommender Systems. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management* (Atlanta, GA, USA) *(CIKM '22)*. 540–550. doi:10.1145/3511808.3557220

[8] Dmytro Gusak, Aleksandra Mojsilovic, Vassilis N. Ioannidis, and Christos Kozyrakis. 2025. Time to Split: Exploring Data Splitting Strategies for Offline Evaluation of Sequential Recommenders. arXiv:2507.16289 [cs.IR] doi:10.48550/arXiv.2507.16289

[9] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4 (2015), 1–19. doi:10.1145/2827872

[10] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1511.06939

[11] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. Adaptive Mixtures of Local Experts. *Neural Computation* 3, 1 (1991), 79–87. doi:10.1162/neco.1991.3.1.79

[12] Dietmar Jannach, Massimo Quadrana, and Paolo Cremonesi. 2022. Session-Based Recommender Systems. In *Recommender Systems Handbook*. Springer. doi:10.1007/978-1-0716-2197-4_8

[13] Yuyi Ji, Aline B. de Oliveira, Raphael D. F. Mesquita, Diego H. C. G. Horta, and Leonardo B. Marinho. 2023. A Critical Study on Data Leakage in Recommender System Offline Evaluation. *ACM Transactions on Information Systems* 41, 3 (2023), 1–27. doi:10.1145/3569930

[14] Michael I. Jordan and Robert A. Jacobs. 1994. Hierarchical Mixtures of Experts and the EM Algorithm. *Neural Computation* 6, 2 (1994), 181–214. doi:10.1162/neco.1994.6.2.181

[15] Wang-Cheng Kang and Julian J. McAuley. 2018. Self-Attentive Sequential Recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. 197–206. doi:10.1109/ICDM.2018.00035

[16] Emanuel Lacic, Markus Reiter-Haas, Dominik Kowald, Manoj Reddy Dareddy, Junghoo Cho, and Elisabeth Lex. 2020. Using Autoencoders for Session-Based Job Recommendations. *User Modeling and User-Adapted Interaction* 30, 4 (2020), 617–658. doi:10.1007/s11257-020-09269-1

[17] Sara Latifi, Dietmar Jannach, and Andrés Ferraro. 2022. Sequential Recommendation: A Study on Transformers, Nearest Neighbors and Sampled Metrics. *Information Sciences* 609 (2022), 660–678. doi:10.1016/j.ins.2022.07.079

[18] Sahar Latifi, Nicola Mauro, and Dietmar Jannach. 2021. Session-Aware Recommendation: A Surprising Quest for the State-of-the-Art. *Information Sciences* 573 (2021), 354–375. doi:10.1016/j.ins.2021.05.048

[19] Jing Li, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Tao Lian, and Jun Ma. 2017. Neural Attentive Session-based Recommendation. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*. 1419–1428. doi:10.1145/3132847.3132926

[20] Jiacheng Li, Yujie Wang, and Julian J. McAuley. 2020. Time Interval Aware Self-Attention for Sequential Recommendation. In *Proceedings of the 13th ACM International Conference on Web Search and Data Mining (WSDM)*. 322–330. doi:10.1145/3336191.3371786

[21] Mingrui Liu, Sixiao Zhang, and Cheng Long. 2025. Facet-Aware Multi-Head Mixture-of-Experts Model for Sequential Recommendation. In *Proceedings of the 18th ACM International Conference on Web Search and Data Mining (WSDM)*. 127–135. doi:10.1145/3701551.3703552

[22] Qiao Liu, Yifu Zeng, Refuoe Mokhosi, and Haibin Zhang. 2018. STAMP: Short-Term Attention/Memory Priority Model for Session-Based Recommendation. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 1831–1839. doi:10.1145/3219819.3219950

[23] Ziwei Liu, Qidong Liu, Yejing Wang, Wanyu Wang, Pengyue Jia, Maolin Wang, Zitao Liu, Yi Chang, and Xiangyu Zhao. 2025. SIGMA: Selective Gated Mamba for Sequential Recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 12 (2025), 12264–12272. doi:10.1609/aaai.v39i12.33336

[24] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H. Chi. 2018. Modeling Task Relationships in Multi-Task Learning with Multi-Gate Mixture-of-Experts. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 1930–1939. doi:10.1145/3219819.3220007

[25] Julian McAuley, Rahul Pandey, and Jure Leskovec. 2015. Inferring Networks of Substitutable and Complementary Products. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 785–794. doi:10.1145/2783258.2783381

[26] Ruihong Qiu, Zi Huang, Hongzhi Yin, and Zijian Wang. 2022. Contrastive Learning for Representation Degeneration Problem in Sequential Recommendation. In *Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM)*. 813–823. doi:10.1145/3488560.3498433

[27] Massimo Quadrana, Alexandros Karatzoglou, Balázs Hidasi, and Paolo Cremonesi. 2017. Personalizing Session-based Recommendations with Hierarchical Recurrent Neural Networks. In *Proceedings of the Eleventh ACM Conference on Recommender Systems (RecSys)*. 130–137. doi:10.1145/3109859.3109896

[28] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing Personalized Markov Chains for Next-Basket Recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*. 811–820. doi:10.1145/1772690.1772773

[29] Retail Rocket. 2015. Retail Rocket Recommender System Dataset. https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset

[30] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=B1ckMDqlg

[31] Yehjin Shin, Jeongwhan Choi, Hyowon Wi, and Noseong Park. 2024. An Attentive Inductive Bias for Sequential Recommendation beyond the Self-Attention. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 8 (2024), 8984–8992. doi:10.1609/aaai.v38i8.28747

[32] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*. 1441–1450. doi:10.1145/3357384.3357895

[33] Hongyan Tang, Junning Liu, Ming Zhao, Xudong Gong, Zhaohui Jin, Yu Lei, Min Zhang, and Xuan Li. 2020. Progressive Layered Extraction (PLE): A Novel Multi-Task Learning (MTL) Model for Personalized Recommendations. In *Proceedings of the 14th ACM Conference on Recommender Systems (RecSys)*. 269–278. doi:10.1145/3383313.3412236

[34] Jiaxi Tang and Ke Wang. 2018. Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding. In *Proceedings of the 11th ACM International Conference on Web Search and Data Mining (WSDM)*. 565–573. doi:10.1145/3159652.3159656

[35] Shoujin Wang, Longbing Cao, Yan Wang, Quan Z. Sheng, Mehmet A. Orgun, and Defu Lian. 2021. A Survey on Session-Based Recommender Systems. *Comput. Surveys* 54, 7 (2021), 1–38. doi:10.1145/3465401

[36] Shoujin Wang, Liang Hu, Yan Wang, Longbing Cao, Quan Z. Sheng, and Mehmet A. Orgun. 2019. Sequential Recommender Systems: Challenges, Progress and Prospects. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*. 6332–6338. doi:10.24963/ijcai.2019/883

[37] Shu Wu, Yuyuan Tang, Yanqiao Zhu, Liang Wang, Xing Xie, and Tieniu Tan. 2019. Session-Based Recommendation with Graph Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence* 33, 1 (2019), 346–353. doi:10.1609/aaai.v33i01.3301346

[38] Yueqi Xie, Peilin Zhou, and Sunghun Kim. 2022. Decoupled Side Information Fusion for Sequential Recommendation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. 1611–1621. doi:10.1145/3477495.3531963

[39] Dingqi Yang, Daqing Zhang, Vincent W. Zheng, and Zhiyong Yu. 2015. Modeling User Activity Preference by Leveraging User Spatial Temporal Characteristics in LBSNs. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 45, 1 (2015), 129–142. doi:10.1109/TSMC.2014.2327053

[40] Shengzhe Zhang, Liyi Chen, Dazhong Shen, Chao Wang, and Hui Xiong. 2025. Hierarchical Time-Aware Mixture of Experts for Multi-Modal Sequential Recommendation. In *Proceedings of the ACM Web Conference 2025 (WWW)*. 3672–3682. doi:10.1145/3696410.3714676

[41] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S. Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, and Xiaofang Zhou. 2019. Feature-level Deeper Self-Attention Network for Sequential Recommendation. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*. 4320–4326. doi:10.24963/ijcai.2019/600

[42] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Y. Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. 2022. Mixture-of-Experts with Expert Choice Routing. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2202.09368

[43] Barret Zoph, Alexander V. Kolesnikov, Neal Wadhwa, Jonathan Huang, Andrew M. Dai, Noam Shazeer, Quoc V. Le, and Yonghui Wu. 2022. Designing Effective Sparse Expert Models. arXiv:2202.08906 [cs.LG] doi:10.48550/arXiv.2202.08906

A Exact RouteRec Instantiation and Sparse Routing Formulation

This appendix records the fixed configuration and the representative cue bank used by RouteRec. The main text emphasizes the conceptual design; this section documents the implementation choices needed for reproducibility and for interpreting the ablations, including the full sparse routing formulation.

A.1 Main Paper Configuration

Table 3: Fixed main configuration used in the paper.

Aspect	Main paper configuration
Backbone layout	self-attn → macro block → mid block → self-attn → micro block
Stage execution	Serial
Routing granularity	Macro and mid: sequence-wise; micro: position-wise
Router composition	Shared hierarchical sparse router at all three stages
Expert activation	Top-3 groups out of 4; top-2 experts per selected group
Active experts	6 out of 12 total per routed decision
History windows	$H = 5$ sessions (macro); $w = 5$ positions (micro)
Core auxiliary terms	Route consistency + z-loss; no load balancing

The placement of the two attention layers reflects the intended temporal roles of the stages. The first attention layer contextualizes the raw prefix before any routing is applied. Macro and mid then operate consecutively because both provide slower sequence-level control over the same contextualized sequence state: macro captures broader recent-history tendency, while mid refines within-session evolution. A second attention layer is inserted only before micro so that the final local router receives a refreshed token interaction state immediately before prediction. This layout was the most stable high-performing architecture in our architecture sweep, and Appendix E compares it against denser and differently ordered alternatives.

This table serves as a reproducibility summary rather than as a search-space description. Architectural choices are fixed across the paper track, whereas capacity and optimization values are selected per dataset from a narrow candidate set and are therefore reported as compact ranges only when no single global value is shared across all reported runs.

A.2 Implementation Specification and Minimal Execution Notes

Table 4: Implementation-level settings used in the selected paper track.

Item	Reported setting
Semantic groups	$G = 4$ groups: Tempo, Focus, Memory, Exposure
Experts per group	$C \in \{2, 3, 4\}$; total experts $E = 4C$
Main sparse setting	$C = 3$, top-3 groups, top-2 experts per selected group
Cue dimensionality	16 scalars per stage
Router hidden width	Representative range 48–128
Routing temperature	Macro 1.0; mid and micro 1.2
Consistency neighbors	$k_{\mathrm{nn}} = 1$ nearest non-self sample per stage
Missing-signal handling	Broadcast when available; zero-fill focus cues if grouping field absent

Minimal execution notes. The implementation uses three simple rules that are easy to miss if one only reads the high-level method description. First, macro and mid cues are computed once per example and then broadcast over the routed positions, whereas micro cues are evaluated at the position level. Second, top-k masking is followed by renormalization inside the active support rather than by leaving the logits in an unnormalized sparse state. Third, if a dataset does not expose the grouping field needed for focus-family cues, the implementation falls back to the fixed missing-signal policy in Table 4 rather than silently redefining the cue family per dataset.

A.3 Representative Behavioral Cue Families

This cue bank is intentionally compact. The cue families are hand-designed in the sense that they encode behavioral hypotheses, but the design goal is to keep them as a small set of summaries that can serve as a stable routing interface across datasets. In practice, the macro bank is instantiated from the recent-session window used by the selected paper instantiation, while mid and micro are computed from the current session prefix at different temporal scopes. The main text therefore reports the family structure and representative examples, while this appendix provides the implementation-oriented view.

B Dataset Analysis and Behavioral Indicators

This section defines the four raw-log indicators used in Table 2 and reports the full dataset-level axis profile. All indicators are computed from training logs before any model training. They are descriptive summaries of recurring behavioral axes in each corpus, not model-specific scores.

Table 5: Representative cue summaries used by RouteRec in the main configuration. Each stage uses four cue families with four scalar summaries per family; the table lists representative examples rather than every implementation detail.

Family	Macro (recent-session tendency)	Mid (within-session evolution)	Micro (short-horizon pressure)
Tempo	Context-valid ratio, last inter-session gap, mean pace, pace trend	Valid-prefix ratio, mean/std interaction interval, session age	Last gap, short-window mean gap, short-window std, deviation from mid-scale pace
Focus	Theme entropy, dominant-theme mass, theme repeat rate, theme shift rate	Category entropy, category top-1 mass, category switch rate, category uniqueness	Immediate category switch, mismatch to recent category history, suffix entropy, suffix uniqueness
Memory	Repeat intensity, adjacent category overlap, adjacent item overlap, repeat trend	Item uniqueness, repeat rate, novelty rate, longest item run	Reconsumption flag, suffix reconsumption rate, suffix item uniqueness, suffix longest run
Exposure	Mean popularity, popularity spread, popularity entropy, popularity trend	Mean popularity, popularity std, popularity entropy, popularity drift	Last-item popularity, suffix popularity spread, suffix popularity entropy, deviation from mid-scale popularity

B.1 Indicator Definitions

Each indicator is a dataset-level scalar derived from raw training logs. All use scale-free (ratio or normalized) forms so that datasets with different interaction volumes remain comparable.

- **Temporal volatility** measures how variable session length is within the corpus. It is the coefficient of variation of session lengths across all sessions:

$$\text{SessVol} = \frac{\text{std}(\text{session_len})}{\text{mean}(\text{session_len})}.$$

  A higher value means session pace and duration are more irregular, which raises the case for tempo-sensitive routing.
- **Transition branching** measures how ambiguous local next-item transitions are. For each source item i, let $P_i(j)$ be the empirical probability of transitioning from i to item j, and let $|\text{supp}(P_i)|$ be the number of distinct observed next items. The normalized next-item entropy for item i is

$$H_{\text{norm}}(i) = \frac{-\sum_j P_i(j) \log P_i(j)}{\log |\text{supp}(P_i)|} \in [0, 1].$$

  The dataset-level score is the transition-count-weighted average over all items with at least two distinct next items:

$$\text{Branch} = \frac{\sum_i n_i \, H_{\text{norm}}(i)}{\sum_i n_i},$$

  where n_i is the number of observed transitions from item i. A higher value means the same local state tends to branch into many different next items, making next-step intent more ambiguous.
- **Memory variability** measures how unevenly repetition behavior is distributed across sessions. For each session, the repeat ratio is repeat_ratio $= 1 -$ (unique items/session length). The dataset-level score is the coefficient of variation of this ratio across all sessions:

$$\text{RepeatVar} = \frac{\text{std}(\text{repeat_ratio})}{\text{mean}(\text{repeat_ratio})}.$$

  A value of zero means all sessions repeat at the same rate. A higher value means some sessions are strongly repeat-heavy while others are not, making repetition a more variable and routing-relevant axis.
- **Context availability** measures how much cross-session history is available for macro-scope routing. It is a composite of three sub-indicators—the fraction of users with at least 2 sessions ($r_{\geq 2}$), the fraction with at least 5 sessions ($r_{\geq 5}$), and a clipped normalized session count $\min(\bar{s}/20,\ 1)$ where $\bar{s}$ is the mean number of sessions per user:

$$\text{CtxAvail} = \tfrac{1}{3}\left(r_{\geq 2} + r_{\geq 5} + \min(\bar{s}/20,\ 1)\right).$$

  A higher value means more users have sufficient cross-session history for macro-scope routing to draw on. This is a support condition for routing, not a cue family.

B.2 Full Dataset-Level Axis Profile

Table 6 extends the four indicators in the main text with additional raw-log summaries: Carryover (mean cross-session item overlap), Drift (fraction of sessions where at least half of items differ from the previous session), and PopConc (popularity concentration, measured as the Gini coefficient of item frequency).

Table 6: Full raw-log axis profile across datasets. All values are computed from training logs before model training. Gain is RouteRec's mean test-score gap from the strongest baseline; WinRate is the fraction of nine reported metrics where RouteRec ranks first.

Dataset	Temporal volatility	Transition branching	Memory variability	Context availability	Carryover	Drift	PopConc	Gain	WinRate
Beauty	0.694	0.843	0.000	0.059	0.000	1.000	0.509	0.0020	0.556
Foursquare	0.944	0.688	1.961	0.988	0.159	0.841	0.647	0.0009	0.556
KuaiRec	0.887	0.903	2.214	0.933	0.003	0.997	0.671	0.0182	1.000
LastFM	0.739	0.630	2.036	0.990	0.072	0.928	0.456	0.0049	1.000
ML-1M	0.381	0.839	0.000	0.264	0.000	1.000	0.699	-0.0031	0.000
Retail Rocket	0.945	0.744	0.858	0.056	0.184	0.816	0.674	0.0055	0.667

The four indicators used in the main text cover distinct recurring behavioral axes: temporal structure (volatility), local transition structure (branching), repetition structure (memory variability), and the support condition for macro-scope routing (context availability). The three remaining columns extend these with cross-session continuity and popularity structure.

B.3 Supplementary Column Definitions

- **Carryover** measures how much item content carries over between consecutive sessions. For each user, let I_m be the item set of session s_m. The Jaccard overlap between adjacent sessions is $|I_{m-1} \cap I_m|/|I_{m-1} \cup I_m|$, and the dataset-level value is the mean over all consecutive session pairs $\mathcal{P}$:

$$\text{Carryover} = \frac{1}{|\mathcal{P}|} \sum_{(m-1,\, m) \in \mathcal{P}} \frac{|I_{m-1} \cap I_m|}{|I_{m-1} \cup I_m|}.$$

  Near-zero values (Beauty, KuaiRec, ML-1M) indicate that consecutive sessions are almost entirely item-disjoint. Retail Rocket and Foursquare show moderate carryover, consistent with return visits to familiar locations or product categories.
- **Drift** is the complement of Carryover: Drift = 1 − Carryover. It reflects the average fraction of cross-session content that is new rather than repeated. A value near 1 means consecutive sessions rarely share items.
- **PopConc** (popularity concentration) measures how unevenly interactions are distributed across the item catalog. It is the Gini coefficient of the empirical item frequency distribution $\{f_i\}$:

$$\text{PopConc} = \frac{\sum_i \sum_j |f_i - f_j|}{2\, N \sum_i f_i},$$

  where N is the number of items. Values range from 0 (uniform distribution) to 1 (all interactions on one item). ML-1M and Retail Rocket have the highest concentration (0.699 and 0.674), reflecting popular movie and product head effects. LastFM is the lowest (0.456), consistent with broader artist listening patterns.

Gain and **WinRate** are result-side summaries, not input-side behavioral indicators. Gain is the mean signed difference between RouteRec's score and the score of the strongest per-metric baseline, averaged over all reported metrics for that dataset. WinRate is the fraction of the nine reported metric values where RouteRec ranks first among all compared methods.

B.4 Directional Rank Correlations

Table 7 reports Spearman rank correlations between each candidate axis and RouteRec's performance measures across the six datasets. Because only six datasets are compared, these values are directional evidence rather than statistical tests. In addition to the axes defined in Sections B.1 and B.3, we include **memory intensity** (RepeatInt), the mean repeat ratio across sessions: RepeatInt = mean(repeat_ratio). It captures the average level of repetition rather than its variability, and is the axis most strongly correlated with gain.

Table 7: Spearman rank correlations between raw-log behavioral axes and RouteRec performance across six datasets. All axis values are computed from training logs before model training.

Behavioral axis	With Gain	With WinRate
Memory intensity (RepeatInt)	**0.841**	0.761
Memory variability (RepeatVar)	0.667	**0.851**
Temporal volatility (SessVol)	0.543	0.441
Carryover	0.377	0.403
Transition branching (Branch)	0.257	−0.088
Context availability (CtxAvail)	−0.086	0.353
Popularity concentration (PopConc)	−0.143	−0.441

Memory variability and memory intensity show the highest rank correlations with gain, consistent with KuaiRec and LastFM (high variability, positive gain) versus Beauty and ML-1M (zero variability, mixed results). Temporal volatility also shows a positive directional correlation. Transition branching is harder to interpret from rank correlation alone: it is highest in KuaiRec where gains are also largest, but similarly high in ML-1M and Beauty where results are more mixed. Context availability shows near-zero correlation with gain but a positive correlation with WinRate, which is consistent with its role as a gating condition: it helps explain why Foursquare and LastFM (both high context availability) benefit even where branching is moderate, rather than predicting the magnitude of gain directly. Popularity concentration shows negative correlations, suggesting that head-heavy exposure alone does not predict routing benefit.

C Dataset, Evaluation, and Full Ranking Results

All experiments were conducted on machines equipped with NVIDIA RTX 3090 24 GB GPUs. Each training or evaluation process used a single GPU; multiple GPUs were used only to run independent experiments in parallel, not for multi-GPU training within a single run.

This appendix summarizes the six datasets and makes the evaluation protocol explicit, since recent work has shown that session construction and split strategy can materially affect offline recommender results [8, 13]. The original releases are cited here for completeness [3, 7, 9, 25, 29, 39]. The dataset descriptions emphasize domain differences and signal availability because the routing branch depends on which behavioral evidence is exposed in each corpus.

C.1 Datasets and Protocol

Table 8: Datasets used in the experiments. Datasets marked with † are sampled processed subsets.

Dataset	Domain	Interactions	Sessions	Items	Avg. len.
KuaiRec†	Video	287,411	24,458	6,477	11.8
Foursquare	POI	145,238	25,369	30,588	5.7
LastFM†	Music	470,408	25,089	52,510	18.8
Retail Rocket	Browse	821,243	153,092	90,211	5.4
Beauty	E-com.	33,488	4,243	3,625	7.9
ML-1M	Movie	575,281	14,539	3,533	39.6

Table 8 reports the statistics of the final processed datasets used in the experiments rather than the raw corpus sizes. In particular, KuaiRec corresponds to the 20% processed subset used in our runs, and LastFM corresponds to the 3% processed subset.

For reproducibility, the protocol is summarized as follows.

(1) Sort all interactions chronologically within each dataset.
(2) Reconstruct sessions with a dataset-specific inactivity threshold.
(3) Iteratively prune sessions with length below 5 and items with support below 3 until the constraints are simultaneously satisfied.
(4) Sort the resulting sessions by session start time.
(5) Assign the earliest 70% of sessions to train and treat the latest 30% as a held-out tail.
(6) Build validation and test from the held-out tail under chronology-preserving cleanup.
(7) Report seen-target metrics as the primary view and keep unseen-target behavior as supplementary accounting.

KuaiRec and Foursquare emphasize stronger short-horizon intent changes, LastFM provides long repeated listening histories, Retail Rocket and Beauty stress sparse revisit behavior, and ML-1M provides longer preference-driven sequences. This spread makes it possible to examine whether behavior-guided routing is primarily beneficial in highly dynamic settings or remains competitive when preference structure is more stable.

Signal availability also differs in a way that matters for routing. KuaiRec offers item, time, and coarse content-side metadata in short-video sessions with strong short-term intent shifts. Foursquare includes item, timestamp, and venue-category structure with bursty check-in dynamics. LastFM provides repeated listening histories with artist-side signals. Retail Rocket is the sparsest side-information setting, relying mainly on anonymous event streams with item and time signals. Beauty exposes item, time, and product-category cues after low-rating removal, while ML-1M contributes longer preference-driven sequences with genre information.

We position our setup as a sessionized sequential evaluation protocol assembled from established components, not as the canonical raw-sequence protocol for sequential recommendation [12, 18, 27]. Sessions are reconstructed post hoc with inactivity heuristics, which is standard practice when logs do not provide explicit session boundaries [12]. The thresholds are adapted to the logging regime of each corpus: Foursquare, KuaiRec, LastFM, ML-1M, and Retail Rocket use a 30-minute inactivity threshold, consistent with dense-log session-aware practice [16, 27], whereas Beauty uses a 14-day threshold because review timestamps are substantially more diffuse. More generally, this threshold choice reflects the observation that sparse and item-poor domains often require longer or more domain-specific notions of a session than dense clickstream data [2]. After sessionization, we repeatedly remove sessions shorter than 5 and items with fewer than 3 interactions until no further pruning is required. This iterative filtering follows the same general rationale as common session-based benchmark preprocessing, where short sessions and very low-support items are filtered to stabilize next-item supervision and evaluation [19, 37].

Session splits are constructed at the session level after sorting by session start time. The earliest 70% of sessions are assigned to training, and the remaining 30% are treated as a held-out tail. This tail is first divided into validation and test using stratification over three factors: whether the target item already appears in the prefix, the target item's popularity bucket with respect to the training split, and the session-length bucket. For the final evaluation files, we then merge the held-out sessions, restore chronological order, and re-split them contiguously into equal validation and test halves. This chronology-preserving design is meant to reduce leakage and keep the offline protocol aligned with temporal deployment order [8, 13].

For the primary evaluation view, we report seen-target metrics only. Concretely, in the held-out files, non-target rows whose items are unseen in training are removed, and unseen targets are retained only for supplementary accounting rather than the main comparison. This matches long-standing session-based and session-aware evaluation practice for ID-based recommendation models, which cannot rank targets outside the training vocabulary and therefore typically separate unseen-target behavior from the primary benchmark table [18, 19, 37]. We report HR, NDCG, and MRR at $k \in \{5, 10, 20\}$ for all methods.

For the result tables, each method is represented by the run with the best mean validation performance over the nine reported metrics under the same selection rule used throughout the paper.

All methods use the same processed train/validation/test partitions. RouteRec's additional macro, mid, and micro cues are deterministic summaries derived from those same observed histories, so the comparison should be read as evaluating different ways of exploiting the shared sessionized interaction context rather than different access to held-out information.

C.2 Model Selection and Tuning Protocol

Because this benchmark is built from a sessionized temporal protocol rather than a canonical raw-sequence leaderboard, the model-selection procedure is stated explicitly. Prior benchmark studies have shown that both baseline quality and model ranking can change materially once methods are re-evaluated under a common framework and retuned under a shared protocol [17, 18]. Accordingly, the comparison does not rely on paper defaults alone.

Our reporting protocol follows three simple rules. First, every model is tuned within a predefined bounded search space that is fixed before the final comparison: learning rate is sampled log-uniformly within a declared dataset-specific interval, while the remaining hyperparameters are drawn from small predeclared candidate sets. Second, these bounded spaces cover a shared core of optimization hyperparameters together with a small number of model-specific controls, so that the search is broad enough to avoid trivial under-tuning but still budgeted tightly enough to remain comparable across methods. Third, the final reported result for every model is the test performance of the configuration that maximizes the mean validation score over the nine reported metrics. When adaptive search is used for computational efficiency, it is restricted to the same predeclared bounds and candidate sets and should therefore be read as an efficient implementation of a controlled bounded search rather than as open-ended retuning [6].

The final paper track therefore uses bounded search spaces rather than unconstrained retuning. In the paper-facing comparison, baselines and RouteRec are aligned to the same shared optimization pool, and model families differ only through a small number of architecture-specific additions. For RouteRec, the selected three-stage routing layout remains fixed, but depth, hidden dropout, and routing-side controls are still tuned within bounded candidate sets. To keep the appendix concise but still reproducible, Table 9 separates the shared optimization pool from the baseline-family and RouteRec-specific additions, and lists the dataset-specific learning-rate intervals explicitly.

C.3 Full Ranking Results

Table 11 restores the full cutoff grid summarized more compactly in Table 1. This presentation preserves the same selection rule as the main text while exposing the complete $k \in \{5, 10, 20\}$ breakdown.

The appendix table confirms that the same qualitative ordering persists across cutoffs: RouteRec is strongest overall, SASRec and TiSASRec remain robust reference points, and the relatively weaker BSARec and DIF-SR results are consistent with the sessionized short-prefix regime discussed in Section 5.2 rather than with a change in selection or evaluation policy.

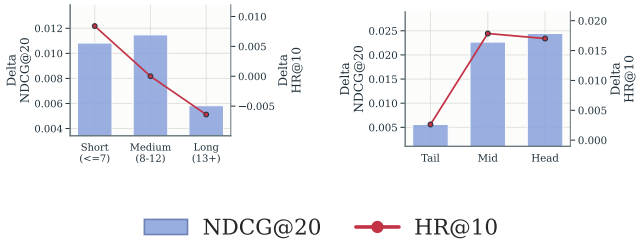
This table makes it possible to verify whether the observed trends persist beyond the compact main-text summary under the same selection rule.

D Where RouteRec Helps Most: Special Bins

This section shows where RouteRec's advantage concentrates within KuaiRec, using session-length and target-frequency partitions. KuaiRec is a representative dataset where behavioral variation and popularity structure are both pronounced, making it a useful lens for within-dataset gain analysis.

Table 10: Special-bin definitions and sample counts used in the appendix gain analysis.

Bin family	Definition rule	Visualization
Session length	Short / medium / long session buckets from the observed prefix length	Figure 6(a)
Target frequency	Rare / mid / frequent target buckets under the train split	Figure 6(b)



(a) Session-length bins (b) Target-frequency bins

Figure 6: KuaiRec special-bin HR@10 gains over the best baseline, partitioned by session length (left) and target-item frequency (right). The shared metric legend appears below the two panels.

The gain pattern is not uniform across all session types. The strongest margin is concentrated in shorter and medium prefixes, which suggests that behavior-guided routing is most effective before the session context becomes long enough for a shared path to dominate. The gain also persists across the rare-to-head target-frequency range, showing that the advantage is not limited to popular items.

Table 9: Bounded candidate grids used in the final paper track.

(a) Shared Optimization Pool

Parameter	Candidate sets
Learning rate	Dataset-specific log-uniform interval; see panel (d)
Weight decay	$\{5 \times 10^{-7}, 10^{-6}, 10^{-5}, 5 \times 10^{-5}, 10^{-4}, 1.5 \times 10^{-4}\}$
Max history length	$\{10, 20, 30\}$ or $\{10, 20, 30, 50\}$
Width / embedding	$\{64, 96, 112, 128, 160\}$ or $\{96, 112, 128, 160, 192\}$
Inner width	$\{128, 192, 224, 256, 320\}$ or $\{192, 224, 256, 320, 384\}$
Layers / heads	Layers $\{1, 2, 3, 4\}$; heads $\{1, 2, 4, 8\}$ if applicable
Dropout	Hidden / ratio: $\{0.10, 0.12, 0.13, 0.15, 0.18\}$ Recurrent: $\{0.10, 0.15, 0.20, 0.25, 0.30\}$ Attention: $\{0.06, 0.08, 0.10, 0.12, 0.15\}$ or $\{0.08, 0.10, 0.12, 0.15, 0.20\}$

(b) Baseline-Family Additions

Model(s)	Additional candidate sets
TiSASRec	Time span $\{64, 128, 256, 384, 512\}$
GRU4Rec	Recurrent dropout $\{0.10, 0.15, 0.20, 0.25, 0.30\}$
DuoRec / FEARec	$\tau\{0.16, 0.18, 0.20, 0.22, 0.24\}$ Contrastive weight $\{0.02, 0.03, 0.04, 0.05, 0.06\}$ Semantic weight $\{0, 0.04, 0.05, 0.08, 0.10\}$ or $\{0.04, 0.05, 0.08, 0.10, 0.12\}$
BSARec	$\alpha\{0.35, 0.50, 0.55, 0.70\}$ $c\{2, 3, 5, 7\}$
DIF-SR / FDSA	Attribute width $\{96, 128, 160, 192\}$ λ_{attr} $\{0.08, 0.09, 0.10, 0.12, 0.14\}$ or $\{0.09, 0.10, 0.12, 0.14, 0.15\}$
DIF-SR	Fusion $\{\text{gate}, \text{sum}, \text{concat}\}$
FAME	Expert count $\{2, 3, 4, 5, 6\}$

(c) RouteRec-Specific Additions

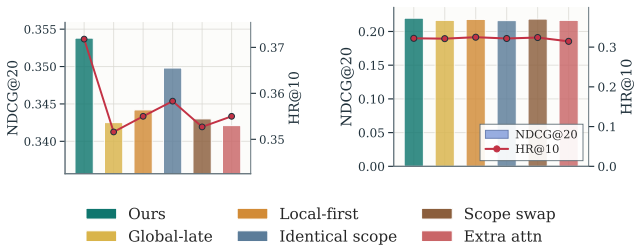
Parameter	Candidate sets
Backbone depth / dropout	Depth: $\{1, 2, 3\}$ Dropout: $\{0.10, 0.12, 0.14, 0.15, 0.16, 0.17\}$ $\{0.18, 0.19, 0.20, 0.22, 0.24\}$
Weight decay	$\{3.125, 5, 6, 10, 15, 20, 50\} \times 10^{-7}$
Routing capacity	Expert scale $\{2, 3, 4\}$ or $\{2, 3, 4, 5\}$ for KuaiRec router width $\{32, 64, 96, 128\}$
Routing dropout	Feature $\{0, 0.03, 0.05, 0.10\}$ attention $\{0.05, 0.06, 0.08, 0.10, 0.12\}$
Regularization	$\lambda_{\text{cons}}\{0, 2.5, 5, 8, 12\} \times 10^{-4}$ $\lambda_z\{0, 5, 10, 20\} \times 10^{-5}$
Cue bank	Cue bank size $\{10, 12, 16, 24\}$ router inner width per cue family (small preset)

(d) Dataset-Specific Learning-Rate Intervals

Dataset	Unified log-uniform LR interval
KuaiRec	3e-4 ∼ 5e-3
Foursquare	1.5e-4 ∼ 2.2e-3
LastFM	8e-5 ∼ 1.2e-3
Retail Rocket	1.5e-4 ∼ 2.2e-3
Beauty	1.5e-4 ∼ 2e-3
ML-1M	1.5e-4 ∼ 2.2e-3

E Extended Framework and Structural Ablations

This section extends the main-text design-justification study with additional temporal-layout controls. The question is whether the selected three-stage RouteRec layout is a stable architectural choice, or whether other ordering and depth configurations perform comparably.



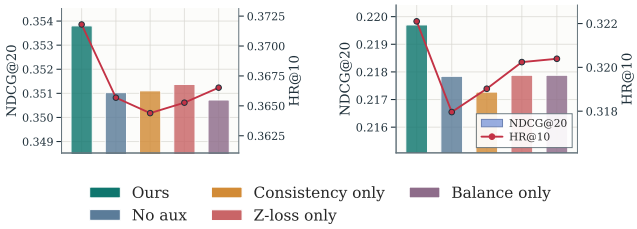
(a) KuaiRec (b) Foursquare

Figure 7: Temporal-layout ablations on KuaiRec (left) and Foursquare (right). Variants include removing individual scopes and substituting parallel weighted-sum configurations for the serial design.

Across both panels, the selected architecture stays near the top without requiring a denser or more redundant arrangement. The remaining controls are useful as negative evidence: changing the scope order or duplicating attention capacity does not yield a cleaner design than the one already used in the paper.

F Objective and Regularization Variants

This section tests whether the selected auxiliary objective is doing all of the work, or whether nearby loss variants perform comparably once the sparse routing design is fixed. The comparison uses the same representative datasets as the main text.



(a) KuaiRec (b) Foursquare

Figure 8: Objective and regularization variants on KuaiRec (left) and Foursquare (right). Variants include removing route consistency, removing z-loss, adding load balancing, and using a standard cross-entropy alone.

The chosen objective still wins, yet the neighboring loss settings stay in the same band. This makes the RouteRec objective look like a tuned refinement rather than an isolated artifact.

G Efficiency Note

RouteRec adds routing overhead relative to a plain SASRec backbone through two sources: cue computation on the routing control path, and the expert-selection mechanism.

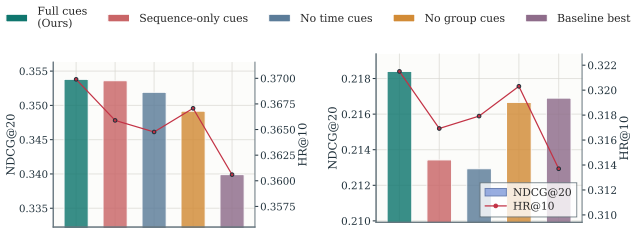
Cue computation involves only scalar statistics (means, standard deviations, entropy values, and popularity lookups) derived from small fixed-size windows. It does not require embedding lookups or attention over the full sequence, so its cost scales with window size rather than with model width. The expert-selection mechanism applies two sparse top-k passes over small weight vectors ($G = 4$ groups, $C \leq 4$ experts per group); at $k_g = 3$ and $k_e = 2$, 6 of 12 experts are active, and the selection itself is a simple masked softmax over a small logit vector.

In practice, the overall training time per epoch is approximately 1.2–1.6× that of SASRec on the same hardware and dataset, depending on sequence length and expert scale. The inference overhead follows the same ratio. The range reflects the difference between shorter sessions (closer to 1.2×, where cue computation is fast relative to backbone attention) and longer sessions (closer to 1.6×, where the backbone dominates and the routing overhead becomes relatively smaller in absolute terms).

RouteRec does not use kernel-level sparse dispatch. The reported overhead is therefore conservative relative to what an optimized production implementation could achieve. Adding more experts or more families increases the overhead, while reducing the number of active families (k_g) or experts per group (k_e) can recover some of it at the cost of coverage.

H Cue Portability

Cue portability matters because item IDs, timestamps, and category labels are not always all available in practice. We therefore retrain RouteRec from scratch under three reduced-cue conditions: removing category cues, removing timestamp cues, or removing both. We compare these variants against the full model and the best baseline.



(a) KuaiRec **(b) Foursquare**

Figure 9: Cue portability on KuaiRec and Foursquare.

Figure 9 shows that reduced-cue variants remain generally competitive with the best baseline on HR@10, even when category or timestamp cues are removed. Each source contributes, and the relative benefit differs between datasets. The full model performs best overall, but the gap between partial-cue and full-cue variants is modest. This suggests that behavior-guided routing does not depend on any single metadata source to remain effective.

Table 11: Full seen-target experimental results across all cutoffs. Datasets marked with † are sampled processed subsets. Each method reports the run with the best mean validation performance under the shared selection rule. Lower Avg. Rank is better.

Dataset	Metric	SASRec	GRU4Rec	TiSASRec	FEARec	DuoRec	BSARec	FAME	DIF-SR	FDSA	RouteRec
KuaiRec†	HR@5	0.3195	0.2770	0.3004	0.3397	0.3270	0.3281	0.3255	0.2275	0.3270	**0.3494**
	HR@10	0.3284	0.3061	0.3216	0.3513	0.3468	0.3359	0.3307	0.2333	0.3453	**0.3669**
	HR@20	0.3379	0.3367	0.3429	0.3688	0.3677	0.3464	0.3430	0.2411	0.3654	**0.3949**
	NDCG@5	0.3148	0.2493	0.2820	0.3249	0.3152	0.3184	0.3172	0.2210	0.3140	**0.3410**
	NDCG@10	0.3177	0.2589	0.2889	0.3286	0.3215	0.3209	0.3188	0.2229	0.3197	**0.3467**
	NDCG@20	0.3200	0.2666	0.2942	0.3330	0.3269	0.3236	0.3220	0.2248	0.3248	**0.3538**
	MRR@5	0.3133	0.2400	0.2759	0.3200	0.3112	0.3152	0.3144	0.2188	0.3097	**0.3382**
	MRR@10	0.3144	0.2441	0.2788	0.3215	0.3138	0.3162	0.3151	0.2196	0.3120	**0.3406**
	MRR@20	0.3150	0.2462	0.2802	0.3227	0.3153	0.3170	0.3160	0.2201	0.3133	**0.3425**
Foursquare	HR@5	0.2350	0.1741	0.2388	0.2403	0.2438	0.1995	0.1954	0.2458	0.2514	**0.2559**
	HR@10	0.3049	0.2180	0.2976	0.3054	0.3195	0.2383	0.2417	0.3032	0.3042	**0.3215**
	HR@20	0.3635	0.2593	0.3501	0.3620	0.3760	0.2773	0.2800	0.3536	0.3524	**0.3776**
	NDCG@5	0.1703	0.1334	0.1735	0.1714	0.1726	0.1483	0.1455	0.1821	**0.1843**	0.1829
	NDCG@10	0.1929	0.1476	0.1926	0.1925	0.1970	0.1609	0.1604	0.2008	0.2014	**0.2042**
	NDCG@20	0.2078	0.1580	0.2059	0.2068	0.2114	0.1708	0.1701	0.2135	0.2136	**0.2184**
	MRR@5	0.1489	0.1199	0.1519	0.1486	0.1491	0.1313	0.1289	0.1611	**0.1621**	0.1588
	MRR@10	0.1583	0.1258	0.1599	0.1574	0.1591	0.1366	0.1351	0.1688	**0.1692**	0.1676
	MRR@20	0.1624	0.1286	0.1635	0.1613	0.1631	0.1393	0.1378	0.1723	**0.1726**	0.1715
LastFM†	HR@5	0.3471	0.2836	0.3405	0.3630	0.3521	0.3312	0.3173	0.3389	0.3494	**0.3693**
	HR@10	0.3775	0.3109	0.3715	0.3876	0.3850	0.3475	0.3359	0.3593	0.3762	**0.3965**
	HR@20	0.3999	0.3338	0.3997	0.4103	0.4101	0.3615	0.3554	0.3825	0.4020	**0.4194**
	NDCG@5	0.2999	0.2588	0.2929	0.3108	0.2959	0.2992	0.2911	0.3103	0.3161	**0.3218**
	NDCG@10	0.3097	0.2676	0.3029	0.3187	0.3066	0.3044	0.2972	0.3169	0.3247	**0.3307**
	NDCG@20	0.3154	0.2735	0.3101	0.3245	0.3129	0.3080	0.3020	0.3228	0.3313	**0.3365**
	MRR@5	0.2841	0.2506	0.2770	0.2933	0.2771	0.2884	0.2822	0.3008	0.3050	**0.3059**
	MRR@10	0.2881	0.2542	0.2812	0.2965	0.2815	0.2906	0.2847	0.3035	0.3085	**0.3096**
	MRR@20	0.2897	0.2558	0.2831	0.2981	0.2833	0.2915	0.2861	0.3051	0.3103	**0.3113**
Retail Rocket	HR@5	0.4610	0.4312	0.4433	0.4730	0.4711	0.4411	0.4398	0.4262	0.4579	**0.4862**
	HR@10	0.5321	0.5048	0.5131	0.5487	0.5478	0.4655	0.4612	0.4322	0.5343	**0.5608**
	HR@20	0.5988	0.5719	0.5810	0.6234	0.6194	0.4887	0.4812	0.4365	0.6053	**0.6315**
	NDCG@5	0.3676	0.3436	0.3528	0.3752	0.3745	0.3877	0.3874	0.3844	0.3600	**0.3907**
	NDCG@10	0.3907	0.3674	0.3755	0.3998	0.3994	0.3957	0.3943	0.3863	0.3848	**0.4148**
	NDCG@20	0.4076	0.3844	0.3926	0.4187	0.4175	0.4015	0.3994	0.3874	0.4028	**0.4327**
	MRR@5	0.3365	0.3145	0.3227	0.3427	0.3423	0.3696	0.3695	**0.3698**	0.3274	0.3588
	MRR@10	0.3461	0.3243	0.3321	0.3529	0.3526	**0.3729**	0.3724	0.3706	0.3377	0.3688
	MRR@20	0.3507	0.3290	0.3368	0.3581	0.3576	**0.3745**	0.3738	0.3709	0.3427	0.3737
Beauty	HR@5	0.1038	0.0573	0.0918	0.1003	0.0993	0.0630	0.0372	0.0993	0.1041	**0.1087**
	HR@10	0.1324	0.0840	0.1235	0.1404	0.1347	0.0726	0.0497	0.1165	0.1318	**0.1561**
	HR@20	0.1637	0.1146	0.1630	0.1757	0.1958	0.0955	0.0592	0.1385	0.1681	**0.2009**
	NDCG@5	0.0707	0.0372	0.0729	0.0669	0.0678	0.0474	0.0263	0.0756	**0.0830**	0.0798
	NDCG@10	0.0800	0.0457	0.0831	0.0797	0.0793	0.0504	0.0303	0.0811	0.0918	**0.0948**
	NDCG@20	0.0879	0.0533	0.0931	0.0889	0.0948	0.0563	0.0327	0.0867	0.1011	**0.1062**
	MRR@5	0.0598	0.0307	0.0666	0.0556	0.0571	0.0422	0.0228	0.0677	**0.0761**	0.0701
	MRR@10	0.0637	0.0341	0.0707	0.0608	0.0619	0.0434	0.0244	0.0700	**0.0796**	0.0762
	MRR@20	0.0659	0.0361	0.0735	0.0634	0.0662	0.0450	0.0250	0.0716	**0.0822**	0.0794
ML-1M	HR@5	0.0858	0.0984	0.0906	0.0906	0.0977	0.0892	0.0968	0.0384	**0.1022**	0.1005
	HR@10	0.1423	0.1456	0.1448	0.1490	0.1555	0.1374	0.1533	0.0669	**0.1606**	0.1568
	HR@20	0.2157	0.2181	0.2220	0.2234	0.2402	0.2018	0.2243	0.1052	**0.2426**	0.2353
	NDCG@5	0.0536	0.0626	0.0582	0.0549	0.0624	0.0568	0.0637	0.0240	**0.0645**	0.0627
	NDCG@10	0.0717	0.0778	0.0756	0.0736	0.0810	0.0723	0.0818	0.0331	**0.0833**	0.0807
	NDCG@20	0.0902	0.0960	0.0950	0.0923	0.1022	0.0884	0.0996	0.0427	**0.1038**	0.1004
	MRR@5	0.0431	0.0510	0.0476	0.0432	0.0508	0.0463	**0.0529**	0.0192	0.0522	0.0504
	MRR@10	0.0505	0.0572	0.0548	0.0508	0.0584	0.0527	**0.0603**	0.0230	0.0600	0.0577
	MRR@20	0.0555	0.0621	0.0600	0.0559	0.0641	0.0570	0.0651	0.0256	**0.0655**	0.0631
Avg. Rank↓		**6.07**	**8.63**	**6.56**	**4.41**	**4.41**	**6.52**	**6.72**	**6.55**	**3.40**	**1.74**